【Solr：企业级搜索应用服务器】

主要内容

1. Solr 简介
2. Solr 搜索原理
3. Solr 单机版安装
4. 可视化管理界面
5. 新建核心
6. 分词
7. Dataimport
8. 使用 SolrJ 操作 Solr
9. Spring Data for Apache Solr
10. SolrCloud

学习目标

知识点	要求
Solr 简介	掌握
Solr 搜索原理	掌握
Solr 单机版安装	掌握
可视化管理界面	掌握
新建核心	掌握
分词	掌握
Dataimport	掌握
使用 SolrJ 操作 Solr	掌握
SolrCloud	掌握
Spring Data for Apache Solr	掌握

一、 Solr 简介

1 为什么使用 Solr

在海量数据下，对 MySQL 或 Oracle 进行模糊查询或条件查询的效率是很低的。而搜索功能在绝大多数项目中都是必须的，如何提升搜索效率是很多互联网项目必须要考虑的问题。

既然使用关系型数据库进行搜索效率比较低，最直接的解决方案就是使用专用搜索工具进行搜索，从而提升搜索效率。

2 常见搜索解决方案

基于 Apache Lucene（全文检索工具库）实现搜索。但是 Lucene 的使用对于绝大多数的程序员都是“噩梦级”的。

基于谷歌 API 实现搜索。

基于百度 API 实现搜索。

3 Solr 简介

Solr 是基于 Apache Lucene 构建的用于搜索和分析的开源解决方案。可提供可扩展索引、搜索功能、高亮显示和文字解析功能。

Solr 本质就是一个 Java web 项目，且内嵌了 Jetty 服务器，所以安装起来非常方便。客户端操作 Solr 的过程和平时我们所写项目一样，就是请求 Solr 中控制器，处理完数据后把结果响应给客户端。

4 正向索引和反向索引

只要讨论搜索就不得不提的两个概念：正向索引（forward index）和反向索引(inverted index)。

正向索引：从文档内容到词组的过程。每次搜索的时候需要搜索所有文档，每个文档比较搜索条件和词组。

文档	词组
I am a chinese	I,am,a,chinses

反向索引：是正向索引的逆向。建立词组和文档的映射关系。通过找到词组就能找到文

档内容。（和新华字典找字很像）

词组	文档
I,am,a,chinses	I am a chinese

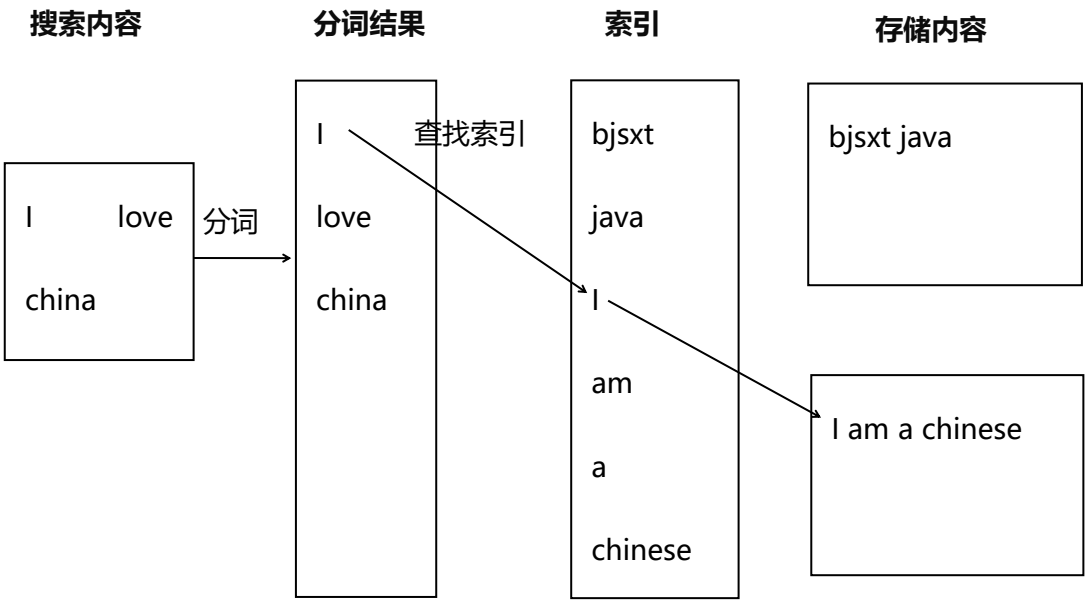
二、 Solr 搜索原理

1 搜索原理

Solr 能够提升检索效率的主要原因就是分词和索引（反向索引）。

分词：会对搜索条件/存储内容进行分词，分成日常所使用的词语。

索引：存储在 Solr 中内容会按照程序员的要求来是否建立索引。如果要求建立索引会把存储内容中关键字（分词）建立索引。



2 Solr 中数据存储说明

Solr 为了给内容建立索引，所以 Solr 就必须具备数据存储能力。所有需要被搜索的内容都需要存储在 Solr 中，在开发中需要把数据库中数据添加到 Solr 中进行初始化，每次修

改数据库中数据还需要同步 Solr 中的数据。

Solr 中数据存储是存储在 Document 对象中，对象中可以包含的属性和属性类型都定义在 schema.xml 中。如果需要自定义属性或自定义属性类型都需要修改 schema.xml 配置文件。从 Solr5 开始 schema.xml 更改名称为 managed-schema(没有扩展名)

三、 Solr 单机版安装

Solr 是使用 Java 编写，所以必选**先安装 JDK**。

1 上传并解压

上传压缩包 solr-8.2.0.tgz 到/usr/local/tmp 中。

解压

cd /usr/local/tmp

tar zxf solr-8.2.0.tgz

2 复制到/usr/local 中

cp -r solr-8.2.0 ../solr

3 Solr 常用目录简介

```
[root@192 solr]# ll
总用量 1680
drwxr-xr-x.  3 root root    147 5月  28 22:44 bin
-rw-r--r--.  1 root root 882555 7月  18 2019 CHANGES.txt
drwxr-xr-x. 12 root root    223 7月  19 2019 contrib
drwxr-xr-x.  4 root root   4096 5月  28 22:44 dist
drwxr-xr-x.  3 root root     38 5月  28 22:44 docs
drwxr-xr-x.  6 root root     88 5月  28 22:44 example
drwxr-xr-x.  2 root root  28672 5月  28 22:44 licenses
-rw-r--r--.  1 root root  12646 7月  18 2019 LICENSE.txt
-rw-r--r--.  1 root root 731038 7月  18 2019 LUCENE_CHANGES.txt
-rw-r--r--.  1 root root  27717 7月  18 2019 NOTICE.txt
-rw-r--r--.  1 root root   7490 7月  18 2019 README.txt
drwxr-xr-x. 10 root root    157 5月  28 22:44 server
```

bin 目录：可执行文件所在目录。

dist 目录：Solr 对外提供的核心、扩展、插件等代码 jar 文件。

server 目录：Solr 搜索应用服务器核心目录。

server/solr：Solr 的索引库所在位置

server/solr-webapp：Solr 的 WAR 核心应用

server/etc：Solr 的核心配置

4 修改启动参数

修改启动参数，否则启动时报警告。提示设置 SOLR_ULIMIT_CHECKS=false

cd /usr/local/solr/bin

vim solr.in.sh

```
# Different limits can be set in solr.in.sh or your profile if you prefer as well.
#SOLR_RECOMMENDED_OPEN_FILES=
#SOLR_RECOMMENDED_MAX_PROCESSES=
SOLR_ULIMIT_CHECKS=false
```

5 启动 Solr

Solr 内嵌 Jetty，直接启动即可。默认监听 8983 端口。

Solr 默认不推荐 root 账户启动，如果是 root 账户启动需要添加-force 参数。

./solr start -force

四、 可视化管理界面

在关闭防火墙的前提下，可以在 windows 的浏览器中访问 Solr。

输入: http://192.168.32.133:8983 就可以访问 Solr 的可视化管理界面。

左侧有 5 个菜单。分别是：

（1）Dashboard：面板显示 Solr 的总体信息。

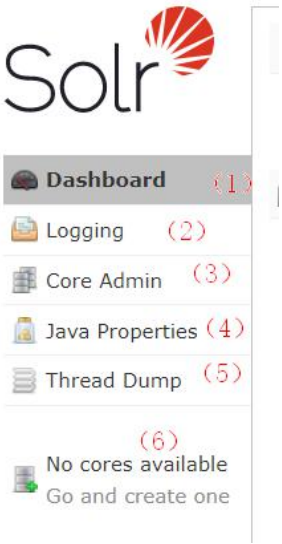
（2）Logging：日志

（3）Core Admin：Solr 的核心。类似于数据的 Database

（4）Java Perperties：所有 Java 相关属性。

（5）Thread Dump：线程相关信息。

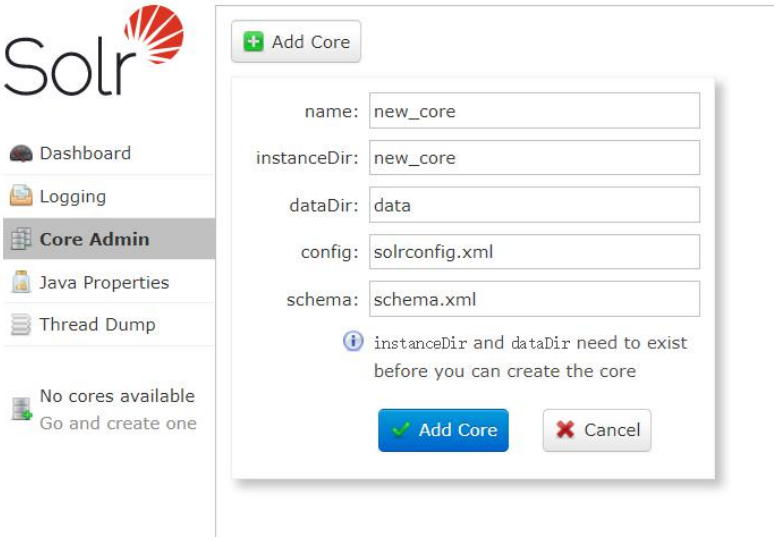
（6）如果有 Core，将显示在此处。



五、 新建核心

Solr 安装完成后默认是没有核心的。需要手动配置。

需要在 solr/server/solr 下新建文件夹，并给定配置文件，否则无法建立。



1 新建目录

在/usr/local/solr/server/solr 中新建自定义名称目录。此处示例名称为 testcore。

cd /usr/local/solr/server/solr

mkdir testcore

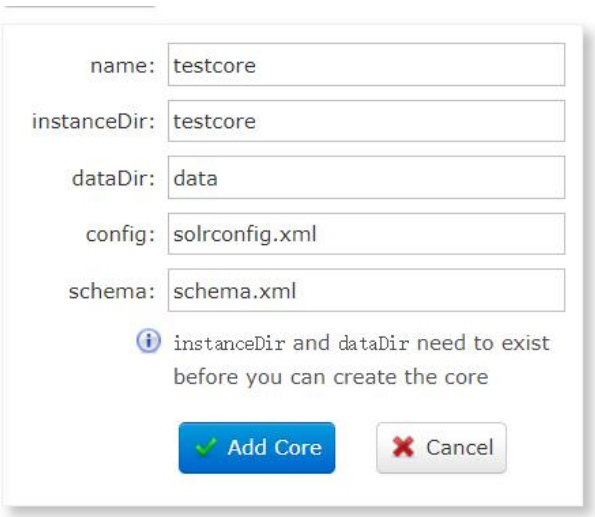
2 复制配置文件

在 configsets 里面包含了_default 和 sample_techproducts_configs。里面都是配置文件示例。_default 属于默认配置，较纯净。sample_techproducts_configs 是带有了一些配置示例。

cp -r configsets/_default/conf/ testcore/

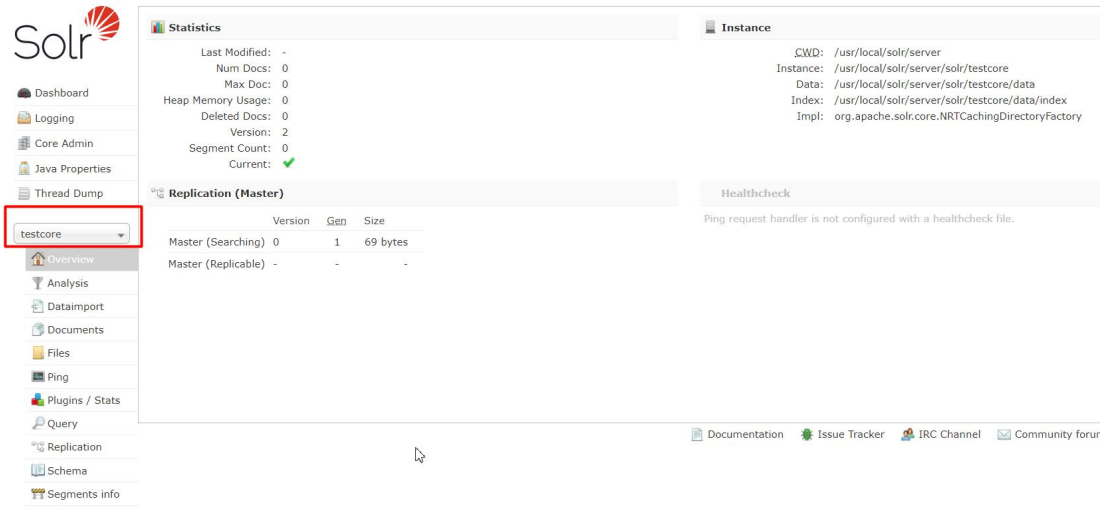
3 填写 Core 信息

在可视化管理界面中 Core Admin 中编写信息后点击 Add Core 后，短暂延迟后 testcore 就会创建成功。schema 处不用更改。



4 出现 testcore

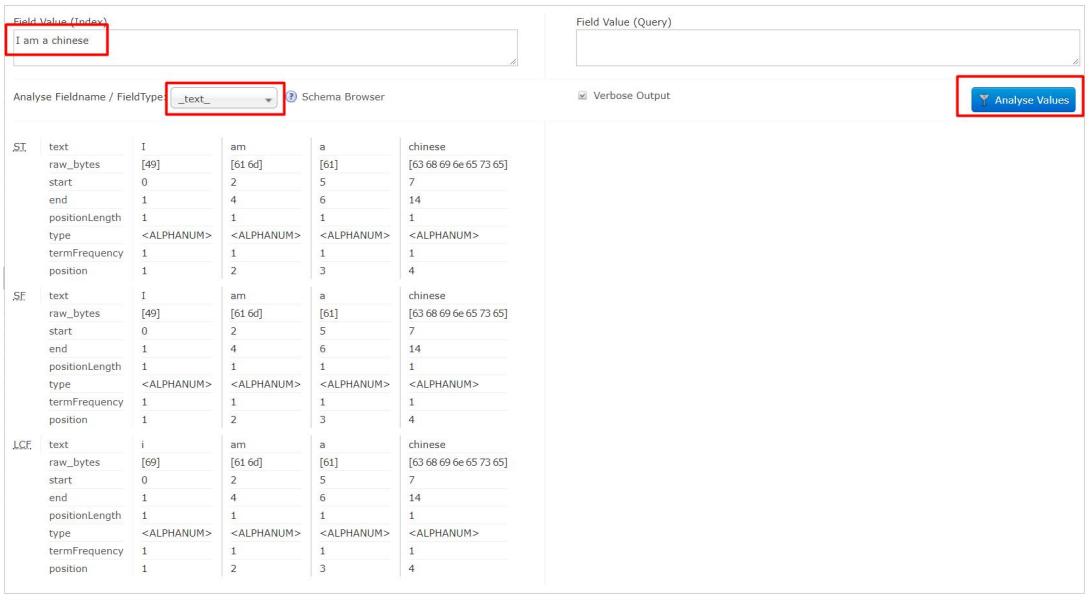
在客户端管理界面中，选择新建的 Core 后，就可以按照自己项目的需求进行操作了。



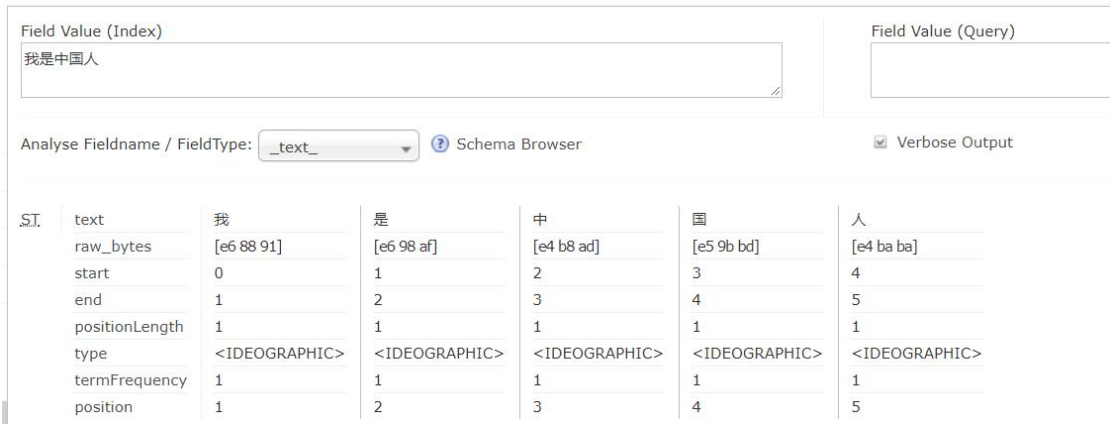
六、 分词 Analysis

在 Solr 可视化管理界面中，Core 的管理菜单项中都会有 Analysis。表示根据 Scheme.xml(managed-schema)中配置要求进行解析。

对英文解析就比较简单了，只要按照空格把英文语句拆分成英文单词即可。



但是如果条件是中文时，把一句话按照字进行拆分就不是很合理了。正确的方式是按照合理的词组进行拆分。



1 中文分词器安装及配置步骤

上传 ik-analyzer.jar 到 webapps 中。

去 https://search.maven.org/search?q=com.github.magese 下载对应版本的 ik-analyzer。可以在"软件/Analyzer"中直接获取。

1.1 上传 jar 到指定目录

上传 ik-analyzer-8.2.0.jar 到

/usr/local/solr/server/solr-webapp/webapp/WEB-INF/lib 目录中

1.2 修改配置文件

修改/usr/local/solr/server/solr/testcore/conf/managed-schema

vim /usr/local/solr/server/solr/testcore/conf/managed-schema

添加下面内容。

排版：Esc 退出编辑状态下：gg=G

```
    <field name="myfield" type="text_ik" indexed="true" stored="true" />
        <fieldType name="text_ik" class="solr.TextField">
                <analyzer type="index">
                        <tokenizer
class="org.wltea.analyzer.lucene.IKTokenizerFactory"    useSmart="false"
conf="ik.conf"/>
                        <filter class="solr.LowerCaseFilterFactory"/>
                </analyzer>
                <analyzer type="query">
                        <tokenizer
class="org.wltea.analyzer.lucene.IKTokenizerFactory"     useSmart="true"
conf="ik.conf"/>
                        <filter class="solr.LowerCaseFilterFactory"/>
                </analyzer>
        </fieldType>
```

1.3 重启

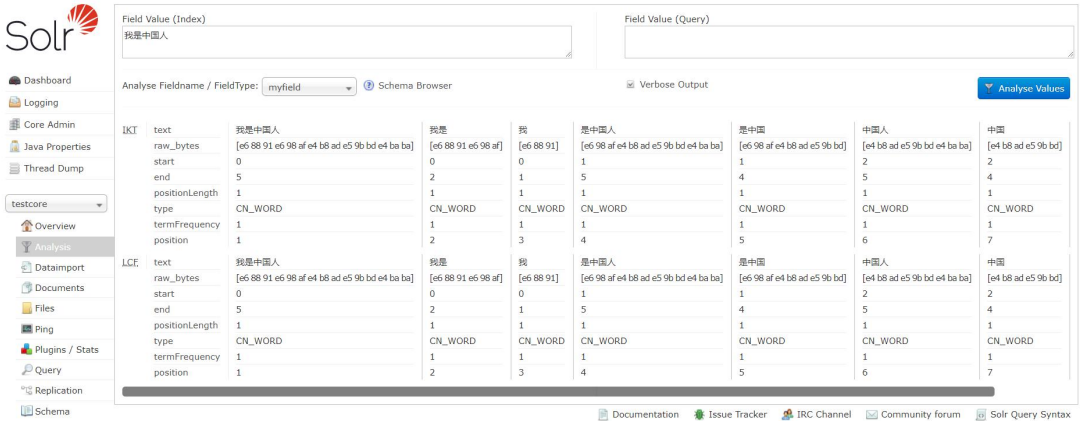
cd /usr/local/solr/bin

./solr stop -all

./solr start -force

1.4 验证

可以在可视化管理界面中找到 myfield 属性进行验证。



2 managed-schema 配置说明

2.1 <fieldType/>

表示定义一个属性类型。在 Solr 中属性类型都是自定义的。在上面配置中 name="text_ik"为自定义类型。当某个属性取值为 text_ik 时 IK Analyzer 才能生效。

2.2 <field/>

表示向 Document 中添加一个属性。

常用属性：

name: 属性名

type:属性类型。所有类型都是 solr 使用<fieldType>配置的

indexed: 是否建立索引

stored: solr 是否把该属性值响应给搜索用户。

required：该属性是否是必须的。默认 id 是必须的。

multiValued：如果为 true，表示该属性为复合属性，此属性中包含了多个其他的

属性。常用在多个列作为搜索条件时，把这些列定义定义成一个新的复合属性，通过搜索一个复合属性就可以实现搜索多个列。当设置为 true 时与<copyField source="" dest=""/>结合使用

2.3 <uniqueKey>

唯一主键，Solr 中默认定义 id 属性为唯一主键。ID 的值是不允许重复的。

2.4 <dynamicField>

名称中允许*进行通配。代表满足特定名称要求的一组属性。

七、 Dataimport

可以使用 Solr 自带的 Dataimport 功能把数据库中数据快速导入到 solr 中.

必须保证 managed-schema 和数据库中表的列对应。

1 修改配置文件

修改 solrconfig.xml，添加下面内容

```
      <!-- 配置数据导入的处理器 -->
      <requestHandler                                    name="/dataimport"
class="org.apache.solr.handler.dataimport.DataImportHandler">
        <lst name="defaults">
      <!--  加载 data-config.xml  -->
          <str name="config">data-config.xml</str>
         </lst>
      </requestHandler>
```

2 新建 data-config.xml

和 solrconfig.xml 同一目录下新建 data-config.xml

```
    <?xml version="1.0" encoding="UTF-8"?>
    <dataConfig>
            <dataSource type="JdbcDataSource"
                    driver="com.mysql.cj.jdbc.Driver"

url="jdbc:mysql://192.168.51.241:3306/maven?serverTimezone=Asia/Shanghai"
                    user="root"
                    password="root"/>
            <document>
                    <entity  name="product"  query="SELECT  id,name,price  from
product">
                            <!--


实现数据库的列和索引库的字段的映射

                     column 指定数据库的列表

                     name  指定索引库的字段名字，必须和 schema.xml 中定义的一样

                     -->
                     <field column="id" name="id"/>
                     <field column="name" name="name"/>
    <field column="price" name="price"/>
            </entity>
     </document>

    </dataConfig>
```

3 添加 jar

向 solr-webapp 中添加三个 jar。在 dist 中两个还有一个数据库驱动。

```
[root@localhost dist]# pwd
/usr/local/solr/dist
[root@localhost dist]# ll
total 10308
-rw-r--r--. 1 root root   28896 Dec 15 02:37 solr-analysis-extras-8.2.0.jar
-rw-r--r--. 1 root root  691951 Dec 15 02:37 solr-analytics-8.2.0.jar
-rw-r--r--. 1 root root   43558 Dec 15 02:37 solr-cell-8.2.0.jar
-rw-r--r--. 1 root root   55874 Dec 15 02:37 solr-clustering-8.2.0.jar
-rw-r--r--. 1 root root 5878401 Dec 15 02:37 solr-core-8.2.0.jar
-rw-r--r--. 1 root root  229140 Dec 15 02:37 solr-dataimporthandler-8.2.0.jar
-rw-r--r--. 1 root root   40883 Dec 15 02:37 solr-dataimporthandler-extras-8.2.0.jar
-rw-r--r--. 1 root root   16056 Dec 15 02:37 solr-jaegertracer-configurator-8.2.0.jar
drwxr-xr-x. 2 root root    4096 Dec 15 02:37 solrj-lib
-rw-r--r--. 1 root root  776411 Dec 15 02:37 solr-langid-8.2.0.jar
-rw-r--r--. 1 root root  135907 Dec 15 02:37 solr-ltr-8.2.0.jar
-rw-r--r--. 1 root root   56439 Dec 15 02:37 solr-prometheus-exporter-8.2.0.jar
-rw-r--r--. 1 root root 2209845 Dec 15 02:37 solr-solrj-8.2.0.jar
-rw-r--r--. 1 root root  333330 Dec 15 02:37 solr-test-framework-8.2.0.jar
-rw-r--r--. 1 root root   31763 Dec 15 02:37 solr-velocity-8.2.0.jar
drwxr-xr-x. 4 root root      54 Dec 15 02:37 test-framework
```

4 操作

重启 solr 后，在可视化管理页面中进行数据导入。

注意：

点击导入按钮后，要记得点击刷新按钮。

八、 菜单项目 Documents 使用办法

以 XML 格式举例

1 新增/修改

当 id 不存在时新增，当 id 存在修改。

```
<doc>
<field name="id">8</field>

<field name="name">明天更大卖</field>

<field name="price">98</field>
</doc>
```

2 删除

2.1 根据主键删除

```
<delete>
```

```
<id>8</id>
</delete>
```

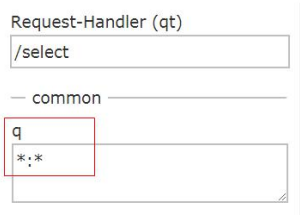
2.2 根据条件删除

```
<delete>
<query>*:*</query>
</delete>
```

九、 菜单项目 query 查询使用办法

1 查询全部

只要在 q 参数中写入*:*既是搜索全部数据。



2 条件查询

在 q 参数部分写入 字段名:搜索条件值， 既是条件搜索

3 分页查询

在条件 start,rows 中输入从第几条数据开始查询，查询多少条数据。下标从 0 开始。类似 MySQL 数据库中的 limit。

4 查询排序

在 sort 条件中输入 字段名 排序规则。 排序规则包括 asc 和 desc。

5 高亮查询

选中 hl 高亮复选框，在 hl.fl 中输入高亮显示的字段名称，在 hl.simple.pre 中输入高亮前缀，在 hl.simple.post 中输入高亮后缀。

十、 使用 SolrJ 操作 Solr

SolrJ 是 Solr 提供的 Java 客户端 API。通过 SolrJ 可以实现 Java 程序对 Solr 中数据的操作。

大前提：添加 SolrJ 依赖。依赖版本和 Solr 版本严格对应

```
<dependencies>
    <dependency>
        <groupId>org.apache.solr</groupId>
        <artifactId>solr-solrj</artifactId>
        <version>8.2.0</version>
    </dependency>
</dependencies>
```

1 新增/修改实现

```
String url = "http://192.168.32.133:8983/solr/testcore";
HttpSolrClient solrClient =  new HttpSolrClient.Builder(url).build();
SolrInputDocument inputDocument = new SolrInputDocument();
inputDocument.addField("id","3");
inputDocument.addField("myfield","myfield3");
solrClient.add(inputDocument);
solrClient.commit();
```

2 删除实现

```
String url = "http://192.168.32.133:8983/solr/testcore";
HttpSolrClient solrClient =  new HttpSolrClient.Builder(url).build();
solrClient.deleteById("3");
solrClient.commit();
```

3 查询实现

```
public void testQuery(){
    try {
        String url = "http://192.168.52.130:8983/solr/testcore";
        HttpSolrClient solrClient = new
HttpSolrClient.Builder(url).build();


        //封装了所有查询条件

        SolrQuery params = new SolrQuery();

        params.setQuery("name:丰富的");

        //排序

        params.setSort("price", SolrQuery.ORDER.desc);

        //分页

        params.setStart(0);
        params.setRows(1);

        //高亮

        params.setHighlight(true);
```

```
        params.addHighlightField("name");
        params.setHighlightSimplePre("<span>");
        params.setHighlightSimplePost("</span>");

        QueryResponse response = solrClient.query(params);
        SolrDocumentList list = response.getResults();

        System.out.println("总条数: "+list.getNumFound());


        //高亮数据

        Map<String, Map<String, List<String>>> highlighting =
response.getHighlighting();

        for(SolrDocument doc :list){
            System.out.println(doc.get("id"));
            Map<String, List<String>> map =
highlighting.get(doc.get("id"));
            List<String> HLList = map.get("name");

            if(HLList!=null&&HLList.size()>0){//显示高亮数据

                System.out.println(HLList.get(0));
            }else{
                System.out.println(doc.get("name"));
            }
            System.out.println(doc.get("price"));
            System.out.println("====================");
        }
        solrClient.close();
    } catch (SolrServerException e) {
        e.printStackTrace();
    } catch (IOException e) {
        e.printStackTrace();
    }
}
```

十一、 Spring Data for Apache Solr

1 Spring Data 简介

Spring Data 是 Spring 的顶级项目。里面包含了 N 多个二级子项目，每个子项目对

应一种技术或工具。其目的为了让数据访问更加简单，更加方便的和 Spring 进行整合。

Spring Data 项目如果单独使用是还需要配置 XML 配置文件的，当和 Spring Boot 整合后使用起来非常方便。spring-boot-starter-data-xx 就是对应的启动器。

2 实现步骤

2.1 添加依赖

```
        <dependencies>
            <dependency>
                <groupId>org.springframework.boot</groupId>
                <artifactId>spring-boot-starter-web</artifactId>
            </dependency>
            <dependency>
                <groupId>org.springframework.boot</groupId>
                <artifactId>spring-boot-starter-test</artifactId>
            </dependency>
            <dependency>
                <groupId>org.springframework.boot</groupId>
                <artifactId>spring-boot-starter-data-solr</artifactId>
            </dependency>
        </dependencies>
```

2.2 编写配置文件

```
    spring:
      data:
        solr:
          host: http://192.168.9.132:8080/solr
          #                                                    zk-host:
192.168.9.132:2181,192.168.9.132:2182,192.168.9.132:2183
```

2.3 编写测试类

```
@RunWith(SpringJUnit4ClassRunner.class)
@SpringBootTest(classes = MyApplication.class)
public class MyTest {
    @Autowired
    private SolrTemplate solrTemplate;
```

```
    public void testInsert(){
        SolrInputDocument doc = new SolrInputDocument();
        doc.setField("id","002");

        doc.setField("item_title","这是一个手机 3");

        UpdateResponse ur = solrTemplate.saveBean("collection1", doc);
        if(ur.getStatus()==0){

            System.out.println("成功");

        }else{

            System.out.println("失败");

        }
        solrTemplate.commit("collection1");
    }

    public void testDelete(){
        UpdateResponse ur = solrTemplate.deleteByIds("collection1",
"change.me");
        if(ur.getStatus()==0){

            System.out.println("成功");

        }else{

            System.out.println("失败");

        }
        solrTemplate.commit("collection1");
    }

    public void query(){
        SimpleQuery query = new SimpleQuery();
        Criteria c = new Criteria("item_keywords");

        c.is("手机");

        query.addCriteria(c);
        query.setOffset(1L);
        query.setRows(1);
        ScoredPage<DemoPojo> sp = solrTemplate.queryForPage("collection1",
query, DemoPojo.class);
        System.out.println(sp.getContent());
    }

    @Test
```

```
    public void queryHL(){

        List<DemoPojo> listResult = new ArrayList<>();

        SimpleHighlightQuery query = new SimpleHighlightQuery();

        //设置查询条件

        Criteria c = new Criteria("item_keywords");

        c.is("手机");

        query.addCriteria(c);

        //分页

        query.setOffset(0L);
        query.setRows(10);

        //排序

        Sort sort = new Sort(Sort.Direction.DESC,"id");
        query.addSort(sort);

        //高亮设置

        HighlightOptions hlo = new HighlightOptions();
        hlo.addField("item_title item_sell_point");
        hlo.setSimplePrefix("<span style='color:red;'>");
        hlo.setSimplePostfix("</span>");
        query.setHighlightOptions(hlo);
        HighlightPage<DemoPojo> hl =
solrTemplate.queryForHighlightPage("collection1", query, DemoPojo.class);
//        System.out.println(hl.getContent());
        List<HighlightEntry<DemoPojo>> highlighted = hl.getHighlighted();
        for(HighlightEntry<DemoPojo> hle : highlighted){
            List<HighlightEntry.Highlight> list = hle.getHighlights();
            DemoPojo dp = hle.getEntity();

            for (HighlightEntry.Highlight h : list){//一个对象里面可能多个

属性是高亮属性

                if(h.getField().getName().equals("item_title")){
                    dp.setItem_title( h.getSnipplets().get(0));
                }
            }
            listResult.add(dp);
        }
        System.out.println(listResult);
```

```
    }
}
```

十二、 SolrCloud

Solr 可以搭建具备容错能力和高可用的 Solr 集群。集群中集群配置、自动负载均衡和查询故障转移、Zookeeper 集群实现集群协调管理，这些全部功能统称为 SolrCloud。

SolrCloud 是基于 Zookeeper 进行管理的。在 Solr 中已经内置了 Zookeeper 相关内容，当执行集群创建命令会自动创建 Zookeeper 相关内容。这个使用的是 Zookeeper 的集群管理功能实现的。

1 搭建

1.1 创建

SolrCloud 已经包含在了 Solr 中，可以直接启动 Solr 集群。

./solr -e cloud -noprompt -force

此命令等同于# ./solr -e cloud -force 全部参数为默认值。

运行成功后会在 example 文件夹多出 cloud 文件夹。

1.2 停止

./solr stop -all

1.3 重新运行

./solr start -c -p 8983 -s ../example/cloud/node1/solr/ -force

#./solr start -c -p 7574 -z localhost:9983 -s ../example/cloud/node2/solr/ -force

1.4 增加集群节点

把 "example/cloud/node1/" 目录中的无效数据删除。只保存 node1/solr/目录中的

solr.xml 和 zoo.cfg 两个配置文件。其他都文件都删除。目录只保存 node1/ 下的 solr 和 logs 两个子目录。

复制 node1 目录。

```
cp -r node1/  node2
cp -r node1/  node3
```

启动多个 Solr 节点，搭建 SolrCloud 集群。

```
# ./solr start -c -p 8983 -s ../example/cloud/node1/solr/ -force
#./solr start -c -p 7574 -z localhost:9983 -s ../example/cloud/node2/solr/ -force
#./solr start -c -p 6666 -z localhost:9983 -s ../example/cloud/node3/solr/ -force
```